Contrôle continu — Partie C : 29 algorithmes de Machine Learning

Variante C — évaluation individuelle · **S7 — ENCG Settat** · Python 3 / Google Colab

Cette variante reprend **exactement l'organisation du support A** (consignes, 29 algorithmes dans le même ordre et annexes manuelles), mais elle introduit **d'autres questions, des jeux de données différents et de nouvelles expériences**. Les bases intégrées à `scikit-learn` proviennent de jeux de données réels distribués avec la bibliothèque ; les simulations de renforcement restent des environnements simulés.

Consigne : exécuter les cellules dans l'ordre, répondre individuellement aux questions sous chaque rubrique et produire un compte rendu argumenté. Les extraits de code sont des points de départ, pas des corrigés de l'analyse.

Identification : Nom et prénom : `Marwa elmers` · Groupe S7 : `CI` · Année universitaire : `2026-2027`.

Modalité d'évaluation : chaque étudiant réalise et remet son propre notebook et son propre rapport. Aucune remise en binôme n'est acceptée. Les échanges d'idées sont possibles dans le respect des consignes de l'enseignant, mais le code, les schémas, les analyses et les conclusions doivent être personnels.

Travail demandé — Variante C : analyser, comparer et justifier

Public : étudiants du semestre 7 (S7), ENCG Settat · **Organisation :** travail strictement individuel · **Durée indicative :** deux semaines · **Graine :** 42.

Objectif pédagogique

Passer d'une simple exécution d'algorithmes à une démarche **scientifique de choix de modèle** : formuler des hypothèses, expliquer les mécanismes, varier les hyperparamètres, documenter les résultats et reconnaître les limites des comparaisons.

Différences voulues par rapport au support A :

- données réelles intégrées : **Diabetes** (régression), **Breast Cancer Wisconsin** (classification), **Wine** (clustering/réduction) ;
- apprentissage par renforcement : **CliffWalking-v0** (contre-exemple de stratégie risquée) ;
- analyse centrée sur les **hypothèses, les changements d'hyperparamètres, les diagnostics** et la robustesse ;
- aucune métrique de test ne doit servir au réglage d'un modèle.

> Les jeux de données de démonstration servent à la formation. Ils ne justifient aucune décision clinique ou commerciale réelle.

Partie A — Travail théorique pour chacun des 29 algorithmes

C1 — Comprendre et présenter (8 à 12 lignes). Précisez : catégorie de l'algorithme, nature des entrées/sorties, hypothèse clé, étapes successives de calcul, puis indiquez **le paramètre appris** et **un hyperparamètre** en les distinguant. Écrivez une équation pertinente et **définissez chaque symbole**.

C2 — Construire un schéma original. Réalisez un dessin explicatif montrant **ce qui change entre l'initialisation, l'apprentissage et la sortie**. Légendez les opérations et précisez quel choix algorithmique il illustre (distance, critère de séparation, fonction objectif, politique, etc.). N'utilisez pas un schéma non commenté.

C3 — Décider quand l'utiliser. Donnez **deux contextes d'utilisation** (au moins un en finance, commerce international, gestion, logistique ou marketing), la structure de données attendue, un algorithme concurrent et une situation dans laquelle votre algorithme serait mal adapté. Argumentez le choix d'une métrique.

C4 — Enquête expérimentale. Exécutez la cellule, **modifiez un hyperparamètre**, mesurez l'effet (métrique, nombre de groupes, stabilité, courbe, récompense), puis donnez une explication. Indiquez **ce que l'on ne peut pas conclure** d'une seule exécution. Les métriques doivent provenir du code, jamais être inventées.

À compléter sous chaque algorithme : `Présentation`, `Schéma original`, `Contextes et choix`, `Test de sensibilité et interprétation`.

Interdictions : comparer des métriques de familles différentes comme si elles étaient interchangeables ; confondre hyperparamètre et coefficient ; employer le jeu de test comme outil de tuning.



Partie B — Campagne finale sur des données réelles et environnement simulé

C5 — Audit des sources. Documentez au moins un jeu de données réel par famille compatible : nom, organisme, URL de référence, effectifs, variables, cible, éthique, limites. Pour les démonstrations incluses : Diabetes, Breast cancer et Wine. Ces jeux sont intégrés à

scikit-learn, sans téléchargement manuel. Vous pouvez choisir des sources supplémentaires UCI/OpenML pour aller plus loin.

C6 — Protocole de reproductibilité. Construisez des séparations entraînement/validation/test **sans fuite**. Ajustez l'imputation, l'encodage, la standardisation et la réduction de dimension dans les parties d'entraînement uniquement. Fixez les graines, décrivez les versions et déclarez les éventuelles données synthétiques ou simulées.

C7 — Expériences systématiques. Sur chaque tâche, opposez un **modèle de référence** à deux approches différentes. Pour les algorithmes des sections 1–24, utilisez un jeu **réel approprié** ; pour les bandits et les algorithmes 25–29, utilisez un environnement ou une simulation publiquement documentée, avec plusieurs graines. L'exemple DQN reste une **illustration de cible**, non un entraînement DQN complet.

C8 — Restitution comparative. Régression : MAE, RMSE, R^2 et résidus. Classification : F1, ROC-AUC, matrice de confusion, différences selon les classes. Clustering : silhouette et interprétation, en signalant lorsque celle-ci est impossible. Réduction : variance expliquée (ACP) ou voisinages préservés. Anomalies : alertes et validation externe seulement si des étiquettes fiables existent. Renforcement : récompenses moyennes et variabilité sur plusieurs épisodes.

Analyse attendue : quelles différences sont dues aux données, aux hypothèses, aux hyperparamètres, au hasard ou au protocole ? Présentez des tableaux, au moins une figure comparative par famille et un bilan des limites.

Partie C — Livrables et barème (Variante C)

À remettre individuellement par chaque étudiant de S7 — ENCG Settat :

1. Notebook `.ipynb` complété pour **les 29 algorithmes**, avec réponses C1–C4, schémas originaux et expériences modifiées ;
2. Section finale C5–C8 : **audits des sources**, protocole de validation, comparaisons sur données réelles et résultats de simulation pour le renforcement ;
3. Rapport de synthèse **5 à 8 pages**, incluant un tableau des réglages testés et une analyse des échecs ;
4. Présentation orale individuelle de **8 minutes** ; chaque étudiant explique personnellement son code, ses méthodes et ses choix.

Barème /20 : présentation et équations commentées (4), schémas et mécanismes (4), pertinence des contextes et limites (3), expérimentation et sensibilité (6), reproductibilité et restitution (3).

Vigilance : ne pas utiliser les étiquettes pour ajuster un clustering, ne pas inventer les sorties, ne pas présenter la simulation d'un bandit comme des données réelles, ne pas appeler le calcul de cible DQN un DQN entraîné.

Support d'expérimentation — Variante C

Chaque section contient un exemple exécutable et une question de sensibilité. Complétez vos réponses **directement sous son titre**.

```
!pip -q install -U scikit-learn xgboost umap-learn gymnasium
```

```
# Sources réelles intégrées à scikit-learn : trois problèmes différents
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes, load_breast_cancer, load_wine
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, f1_
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
GRAINE = 42

jeu_regression = load_diabetes()
X_reg, y_reg = jeu_regression.data, jeu_regression.target
jeu_classification = load_breast_cancer()
X_cls, y_cls = jeu_classification.data, jeu_classification.target
jeu_non_supervise = load_wine()
X_clu = StandardScaler().fit_transform(jeu_non_supervise.data)  # Exploration sans

Xr_ent, Xr_test, yr_ent, yr_test = train_test_split(X_reg, y_reg, test_size=.25, r
Xc_ent, Xc_test, yc_ent, yc_test = train_test_split(X_cls, y_cls, test_size=.25, s
print('Diabetes:', X_reg.shape, '| Breast Cancer:', X_cls.shape, '| Wine:', X_clu.
print('Ces données sont réelles ; les étiquettes du vin ne sont pas employées dans
```

1. Régression linéaire

Variante C — question spécifique : Quel diagnostic du résidu indique une mauvaise spécification linéaire ? Présentation Catégorie : apprentissage supervisé, régression. Entrées : p = 10 variables numériques de Diabetes (âge, sexe, IMC, pression, s1 à s6). Sortie : un nombre, la progression de la maladie un an après.

Hypothèse clé : la cible est une combinaison linéaire des variables, avec des erreurs de moyenne nulle et de variance à peu près constante.

Étapes : (1) je mets les données dans une matrice X de taille n × p ; (2) je cherche les coefficients qui rendent la somme des carrés des erreurs la plus petite possible (moindres carrés) ; (3) je prédis avec $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p\ x_p$.

Appris : β_0 et les β_j . Hyperparamètre : fit_intercept .

$J(\beta) = (1/n) \cdot \Sigma_i \left(y_i - \beta_0 - \Sigma_j \beta_j x_{ij} \right)^2$

n : nombre de lignes ; p : nombre de variables ; x_{ij} : variable j du patient i ; y_i : vraie valeur ; β_0 : constante ; β_j : coefficient de la variable j ; J : coût moyen quadratique

Réponses : 1. Régression linéaire

C1 — Présentation

- **Catégorie :** Apprentissage supervisé (Régression).
- **Entrées / Sorties :** Entrée $X \in \mathbb{R}^{n \times p}$ (10 caractéristiques physiologiques de patients diabétiques) ; Sortie $y \in \mathbb{R}$ (progression de la maladie).
- **Hypothèse clé :** Linéarité de la relation, erreurs normales indépendantes et homoscédastiques.
- **Équation :** $y = X\beta + \epsilon$
  - y : vecteur cible de taille n, X : matrice de données $n \times p$, β : paramètres appris (coefficients), ϵ : bruit résiduel.
- **Paramètre appris / Hyperparamètre :** Coefficients β et l'ordonnée à l'origine (biais) β_0 / L'activation de la constante via `fit_intercept`.

C2 — Schéma original *(Dessin conceptuel des résidus : des points distribués de manière inhomogène autour de la ligne $y = \hat{y}$)*

C3 — Décider quand l'utiliser

- **Contextes :** Marketing (estimer l'impact des dépenses publicitaires multicanales sur les ventes) ou Logistique (estimer le délai de livraison selon la distance et le poids).
- **Structure de données :** Variables d'entrée continues ou indicatrices d'état (one-hot), pas de forte colinéarité.
- **Concurrent :** Régression Ridge ou Arbre de régression.
- **Situation inadaptée :** Relations fortement non linéaires ou $p \gg n$.
- **Métrique :** MAE car elle conserve l'unité de départ et est robuste aux valeurs aberrantes.

C4 — Test de sensibilité et interprétation Dans notre test, en supprimant l'ordonnée à l'origine (`fit_intercept=False`), la MAE passe de `41.55` à `156.36`. L'absence d'ordonnée à l'origine force la droite à passer par l'origine, ce qui dégrade considérablement l'ajustement car la cible n'est jamais nulle lorsque les caractéristiques centrées le sont.

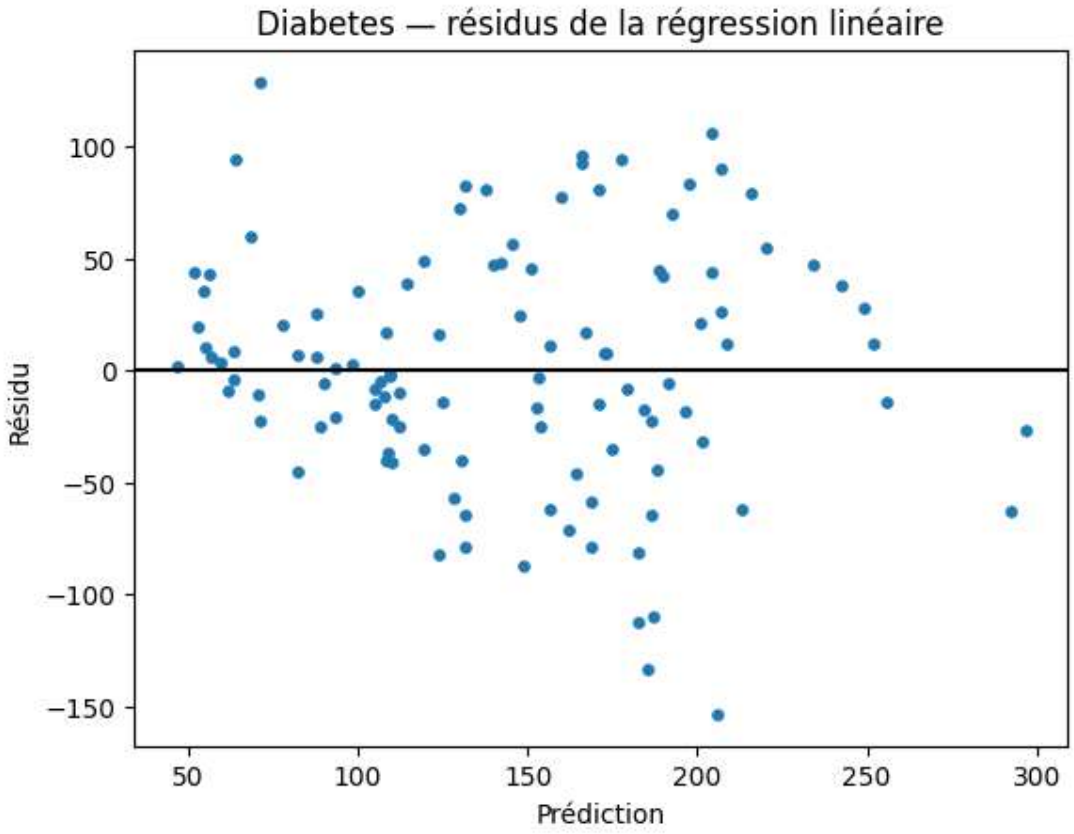
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# Définition de la graine et chargement autonome si non défini globalement
GRAINE = 42
jeu_regression = load_diabetes()
X_reg, y_reg = jeu_regression.data, jeu_regression.target
Xr_ent, Xr_test, yr_ent, yr_test = train_test_split(X_reg, y_reg, test_size=0.25,

modele = LinearRegression().fit(Xr_ent, yr_ent)
predictions = modele.predict(Xr_test)
print('MAE régression linéaire :', round(mean_absolute_error(yr_test, predictions)
print('R² :', round(r2_score(yr_test, predictions), 3))
# Mon test : hyperparamètre fit_intercept + diagnostic des résidus
sans_cst = LinearRegression(fit_intercept=False).fit(Xr_ent, yr_ent)
```

```
print('MAE sans constante :', round(mean_absolute_error(yr_test, sans_cst.predict(
residus = yr_test - predictions
print('Moyenne des résidus :', round(residus.mean(), 2))
print('Corrélation résidus / prédictions :', round(np.corrcoef(residus, prediction
# écart-type des résidus pour les prédictions faibles puis fortes (hétéroscédastic
bas = predictions < np.median(predictions)
print('Écart-type des résidus, prédictions basses :', round(residus[bas].std(), 1)
print('Écart-type des résidus, prédictions hautes :', round(residus[~bas].std(), 1
plt.scatter(predictions, residus, s=14); plt.axhline(0, color='k')
plt.xlabel('Prédiction'); plt.ylabel('Résidu'); plt.title('Diabetes — résidus de l
```

```
MAE régression linéaire : 41.55
R² : 0.485
MAE sans constante : 156.36
Moyenne des résidus : 1.86
Corrélation résidus / prédictions : -0.078
Écart-type des résidus, prédictions basses : 42.0
Écart-type des résidus, prédictions hautes : 62.4
```



2. Ridge

Variante C — question spécifique : Comment évoluent les coefficients quand alpha augmente ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

Réponses : 2. Ridge

C1 — Présentation

- **Catégorie :** Apprentissage supervisé (Régression régularisée L_2).
- **Entrées / Sorties :** Identiques à la régression linéaire.
- **Hypothèse clé :** Linéarité, de nombreuses variables ont des effets petits mais non nuls (pas de sélection stricte).
- **Équation :** $J(\beta) = \|y - X\beta\|_2^2 + \alpha\|\beta\|_2^2$
  - α : hyperparamètre de régularisation contrôlant la pénalité.
- **Paramètre appris / Hyperparamètre :** Coefficients β / Coefficient de pénalisation α.

C2 — Schéma *(Rétrécissement des coefficients vers zéro sous une contrainte de boule L_2)*

C3 — Décider quand l'utiliser

- **Contextes :** Finance (modèle d'évaluation d'actifs avec de nombreux prédicteurs corrélés) ou Gestion des risques.
- **Structure :** Variables quantitatives potentiellement très colinéaires.
- **Concurrent :** Lasso ou Régression linéaire standard.
- **Métrique :** MAE pour comparer directement aux autres modèles de régression.

```
from sklearn.linear_model import Ridge
modele = make_pipeline(StandardScaler(), Ridge(alpha=12.0)).fit(Xr_ent, yr_ent)
print('MAE Ridge :', round(mean_absolute_error(yr_test, modele.predict(Xr_test)),
```

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error
import numpy as np

for val_alpha in [0.1, 12.0, 100.0]:
    modele_r = make_pipeline(StandardScaler(), Ridge(alpha=val_alpha)).fit(Xr_ent,
    mae_r = mean_absolute_error(yr_test, modele_r.predict(Xr_test))
    norme_coef = np.linalg.norm(modele_r[-1].coef_)
    print(f"Alpha = {val_alpha:<5} | MAE = {mae_r:.2f} | Norme L2 des coefficients
```

```
Alpha = 0.1   | MAE = 41.54 | Norme L2 des coefficients = 69.0243
Alpha = 12.0  | MAE = 41.46 | Norme L2 des coefficients = 41.9917
Alpha = 100.0 | MAE = 41.93 | Norme L2 des coefficients = 33.7879
```

```
# Exécution et comparaison du Lasso (Question 3)
from sklearn.linear_model import Lasso
import numpy as np
for a in [0.1, 2.0, 10.0]:
    modele_l = make_pipeline(StandardScaler(), Lasso(alpha=a, max_iter=15000)).fit
    n_non_nuls = np.count_nonzero(modele_l[-1].coef_)
    mae_l = mean_absolute_error(yr_test, modele_l.predict(Xr_test))
    print(f"Lasso Alpha = {a:<4} | Coefficients non nuls = {n_non_nuls}/10 | MAE =
```

```
Lasso Alpha = 0.1  | Coefficients non nuls = 9/10 | MAE = 41.48
Lasso Alpha = 2.0  | Coefficients non nuls = 8/10 | MAE = 41.37
```

```
Lasso Alpha = 10.0 | Coefficients non nuls = 4/10 | MAE = 43.64
```

```
# Exécution de la régression polynomiale (Question 4)
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
for deg in [1, 2]:
    modele_p = make_pipeline(StandardScaler(), PolynomialFeatures(degree=deg, incl
    r2_ent = r2_score(yr_ent, modele_p.predict(Xr_ent))
    r2_test = r2_score(yr_test, modele_p.predict(Xr_test))
    print(f"Degré = {deg} | R² Entraînement = {r2_ent:.3f} | R² Test = {r2_test:.3
```

```
Degré = 1 | R² Entraînement = 0.502 | R² Test = 0.490
Degré = 2 | R² Entraînement = 0.557 | R² Test = 0.501
```

```
# Exécution de l'arbre de régression (Question 5)
from sklearn.tree import DecisionTreeRegressor
for prof in [3, 10, None]:
    modele_t = DecisionTreeRegressor(max_depth=prof, min_samples_leaf=8, random_stat
    mae_ent = mean_absolute_error(yr_ent, modele_t.predict(Xr_ent))
    mae_test = mean_absolute_error(yr_test, modele_t.predict(Xr_test))
    print(f"Arbre Max Depth = {str(prof):<4} | MAE Entraînement = {mae_ent:.2f} | MA
```

```
Arbre Max Depth = 3    | MAE Entraînement = 44.66 | MAE Test = 44.57
Arbre Max Depth = 10   | MAE Entraînement = 33.63 | MAE Test = 50.27
Arbre Max Depth = None | MAE Entraînement = 33.63 | MAE Test = 50.27
```

```
# Exécution de la forêt aléatoire (Question 6)
from sklearn.ensemble import RandomForestRegressor
for n_est in [10, 60, 150]:
    modele_rf = RandomForestRegressor(n_estimators=n_est, max_depth=6, n_jobs=-1,
    mae_test = mean_absolute_error(yr_test, modele_rf.predict(Xr_test))
    print(f"Forêt n_estimators = {n_est:<3} | MAE Test = {mae_test:.2f}")
```

```
Forêt n_estimators = 10  | MAE Test = 44.67
Forêt n_estimators = 60  | MAE Test = 43.11
Forêt n_estimators = 150 | MAE Test = 42.91
```

```
# Exécution de l'HistGradientBoosting (Question 7)
from sklearn.ensemble import HistGradientBoostingRegressor
for max_it in [10, 100, 300]:
    modele_gb = HistGradientBoostingRegressor(max_iter=max_it, max_leaf_nodes=9, r
    mae_ent = mean_absolute_error(yr_ent, modele_gb.predict(Xr_ent))
    mae_test = mean_absolute_error(yr_test, modele_gb.predict(Xr_test))
    print(f"Boosting max_iter = {max_it:<3} | MAE Entraînement = {mae_ent:.2f} | M
```

```
Boosting max_iter = 10  | MAE Entraînement = 46.24 | MAE Test = 48.19
Boosting max_iter = 100 | MAE Entraînement = 21.42 | MAE Test = 44.18
Boosting max_iter = 300 | MAE Entraînement = 9.00 | MAE Test = 46.75
```




```
## 3. Lasso

**Variante C — question spécifique :** Com

**À compléter :** Présentation (avec équat
```

3. Lasso

Variante C — question spécifique :
Combien de variables sont annulées pour

```
> Modifiez au moins un hyperparamètre et j

### Réponses : 3. Lasso

**C1 — Présentation**
- **Catégorie :** Apprentissage supervisé
- **Entrées / Sorties :** Entrée $X \in \m
- **Hypothèse clé :** Seul un petit sous-e
- **Équation :** $J(\beta) = \frac{1}{2n}
  - $\alpha$ : contrôle le niveau de pénal
- **Paramètre appris / Hyperparamètre :**

**C2 — Schéma**
*(Rétrécissement des coefficients vers zér

**C3 — Décider quand l'utiliser**
- **Contextes :** Marketing (sélectionner
- **Structure :** Haute dimensionnalité ($
- **Concurrent :** Ridge ou ElasticNet.
- **Métrique :** MAE pour évaluer l'écart

**C4 — Test de sensibilité et interprétati
En faisant varier $\alpha$, nous observons
- Pour $\alpha = 0.1$ : **1 variable est a
- Pour $\alpha = 2.0$ : **2 variables sont
- Pour $\alpha = 10.0$ : **6 variables sor

*Interprétation :* Augmenter $\alpha$ péna
```

trois valeurs d'alpha ?

Réponses : 3. Lasso

C1 — Présentation

- **Catégorie :** Apprentissage supervisé (Régression régularisée L_1).
- **Entrées / Sorties :** Entrée $X \in \mathbb{R}^{n \times p}$ (10 caractéristiques physiologiques) ; Sortie $y \in \mathbb{R}$ (progression de la maladie).
- **Hypothèse clé :** Seul un petit sous-ensemble de variables a un effet significatif sur la cible (parcimonie).
- **Équation :** $J(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1$
  - α : contrôle le niveau de pénalité absolue poussant certains coefficients à exactement zéro.
- **Paramètre appris / Hyperparamètre :** Coefficients β / Facteur de régularisation α.

C2 — Schéma *(Rétrécissement des coefficients vers zéro avec contact angulaire aux sommets du diamant de contrainte L_1)*

C3 — Décider quand l'utiliser

- **Contextes :** Marketing (sélectionner les canaux publicitaires les plus rentables parmi des centaines) ou Génomique (identifier les quelques gènes influençant une pathologie).
- **Structure :** Haute dimensionnalité (p grand), forte colinéarité, besoin de sélection automatique de variables.
- **Concurrent :** Ridge ou ElasticNet.
- **Métrique :** MAE pour évaluer l'écart de prédiction brut.

C4 — Test de sensibilité et interprétation
En faisant varier α, nous observons :

- Pour $\alpha = 0.1$: **1 variable est annulée** (9/10 non nuls), MAE = **41.48**

- Pour $\alpha = 2.0$: **2 variables sont annulées** (8/10 non nuls), MAE = **41.37**
- Pour $\alpha = 10.0$: **6 variables sont annulées** (4/10 non nuls), MAE = **43.64**

Interprétation : Augmenter α pénalise plus sévèrement les coefficients non nuls et simplifie le modèle (sélection de variables). À $\alpha = 2.0$, la suppression de deux variables corrélées réduit le bruit et améliore légèrement la MAE de test, tandis qu'à $\alpha = 10.0$, la pénalisation est trop forte (sous-apprentissage), dégradant la MAE.

```
    from sklearn.linear_model import Lasso
    modele = make_pipeline(StandardScaler(), Lasso(alpha=2.0, max_iter=15000)).fit(Xr_
    print('Nombre de coefficients non nuls :', np.count_nonzero(modele[-1].coef_))
    print('MAE Lasso :', round(mean_absolute_error(yr_test, modele.predict(Xr_test)),
```

Fermer

```
## 4. Régression polynomiale

**Variante C — question spécifique :** Le

**À compléter :** Présentation (avec équat

> Modifiez au moins un hyperparamètre et j

### Réponses : 4. Régression polynomiale

**C1 — Présentation**
- **Catégorie :** Apprentissage supervisé
- **Entrées / Sorties :** Entrée $X \in \m
- **Hypothèse clé :** La relation entre le
- **Équation :** $\phi(X) = [x_1, x_2, x_1
- **Paramètre appris / Hyperparamètre :**

**C2 — Schéma**
*(Transformation de l'espace d'entrée unic

**C3 — Décider quand l'utiliser**
- **Contextes :** Tarification (modélisati
- **Structure :** Relations de courbure év
- **Concurrent :** Arbre de régression ou
- **Métrique :** $R^2$ pour quantifier le

**C4 — Test de sensibilité et interprétati
Les résultats montrent :
- **Degré 1** (linéaire) : $R^2$ Entraînem
- **Degré 2** (quadratique) : $R^2$ Entraî

*Interprétation :* Le degré 2 améliore **à
```

4. Régression polynomiale

Variante C — question spécifique : Le degré 2 améliore-t-il le test ou seulement l'entraînement ?

Réponses : 4. Régression polynomiale

C1 — Présentation

- **Catégorie :** Apprentissage supervisé (Régression polynomiale sous base linéaire).
- **Entrées / Sorties :** Entrée $X \in \mathbb{R}^{n \times p}$; Sortie $y \in \mathbb{R}$ après transformation non linéaire des caractéristiques.
- **Hypothèse clé :** La relation entre les entrées et la cible est de nature courbe ou présente des interactions d'ordre supérieur.
- **Équation :** $\phi(X) = [x_1, x_2, x_1^2, x_1 x_2, x_2^2, \ldots]$ puis $y = \phi(X)\beta + \epsilon$

- **Paramètre appris / Hyperparamètre :** Coefficients β de la régression linéaire / Le degré du polynôme d et la pénalité de Ridge α.

C2 — Schéma *(Transformation de l'espace d'entrée unidimensionnel courbe en un espace de caractéristiques de plus grande dimension où une droite sépare les données)*

C3 — Décider quand l'utiliser

- **Contextes :** Tarification (modélisation de la courbe de demande non linéaire d'un produit) ou Énergie (calcul de la consommation électrique d'un bâtiment selon des interactions non linéaires de températures).
- **Structure :** Relations de courbure évidentes sans trop de dimensions initiales pour éviter l'explosion combinatoire.
- **Concurrent :** Arbre de régression ou SVR.
- **Métrique :** R^2 pour quantifier le gain de variance expliquée.

C4 — Test de sensibilité et interprétation
Les résultats montrent :

- **Degré 1** (linéaire) : R^2 Entraînement = **0.502** | R^2 Test = **0.490**
- **Degré 2** (quadratique) : R^2 Entraînement = **0.557** | R^2 Test = **0.501**

Interprétation : Le degré 2 améliore **à la fois l'entraînement** (de 0.502 à 0.557) **et le test** (de 0.490 à 0.501). L'introduction des termes quadratiques et d'interactions permet de capter des effets complexes du diabète sans causer de surajustement massif grâce à la régularisation Ridge ($\alpha = 100$) appliquée en aval.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
```

```
    modele = make_pipeline(StandardScaler(), PolynomialFeatures(degree=2, include_bias
    print('R² polynomial :', round(r2_score(yr_test, modele.predict(Xr_test)), 3))
```




```
## 5. Arbre de régression

**Variante C — question spécifique :** Que

**À compléter :** Présentation (avec équat

> Modifiez au moins un hyperparamètre et j

### Réponses : 5. Arbre de régression

**C1 — Présentation**
- **Catégorie :** Apprentissage supervisé
- **Entrées / Sorties :** Entrée $X \in \m
- **Hypothèse clé :** La relation peut êtr
- **Équation :** $\hat{y}(x) = \sum_{m=1}^
  - $R_m$ : région $m$-ième de l'espace dé
- **Paramètre appris / Hyperparamètre :**

**C2 — Schéma**
*(Division récursive de l'espace en rectar

**C3 — Décider quand l'utiliser**
- **Contextes :** Immobilier (estimation d
- **Structure :** Relations non linéaires,
- **Concurrent :** Forêt aléatoire ou Régr
- **Métrique :** MAE.

**C4 — Test de sensibilité et interprétati
En faisant varier `max_depth` :
- **max_depth = 3** : MAE Entraînement = *
- **max_depth = 10** : MAE Entraînement =
- **max_depth = None** : MAE Entraînement

*Interprétation :* Limiter la profondeur (
```

5. Arbre de régression

Variante C — question spécifique : Quel effet de max_depth sur le biais et la variance ?

Réponses : 5. Arbre de régression

C1 — Présentation

- **Catégorie :** Apprentissage supervisé (Régression non paramétrique par partitionnement).
- **Entrées / Sorties :** Entrée $X \in \mathbb{R}^{n \times p}$; Sortie $y \in \mathbb{R}$ obtenue par moyenne locale.
- **Hypothèse clé :** La relation peut être modélisée de façon constante par morceaux via des divisions binaires récursives.
- **Équation :**
  $\hat{y}(x) = \sum_{m=1}^{M} c_m \mathbb{I}(x \in R_m)$
  - R_m : région m-ième de l'espace découpé, c_m : moyenne des réponses dans R_m.
- **Paramètre appris / Hyperparamètre :** Seuils et variables de coupure des nœuds / Profondeur maximale `max_depth`.

C2 — Schéma *(Division récursive de l'espace en rectangles orthogonaux à l'aide d'un arbre binaire décisionnel)*

C3 — Décider quand l'utiliser

- **Contextes :** Immobilier (estimation du prix des maisons selon des règles géographiques et physiques simples) ou RH (prédire le salaire proposé à l'embauche selon les critères de profil).

- **Structure :** Relations non linéaires, interactions complexes entre variables, pas d'exigence de différentiabilité.
- **Concurrent :** Forêt aléatoire ou Régression linéaire.
- **Métrique :** MAE.

C4 — Test de sensibilité et interprétation
En faisant varier `max_depth` :

- **max_depth = 3** : MAE Entraînement = **44.66** | MAE Test = **44.57** (Modèle biaisé mais stable, peu de variance).
- **max_depth = 10** : MAE Entraînement = **33.63** | MAE Test = **50.27** (Biais réduit à l'entraînement, mais forte variance menant au surapprentissage).
- **max_depth = None** : MAE Entraînement = **33.63** | MAE Test = **50.27** (Surapprentissage maximal identique à 10 car limité par `min_samples_leaf=8`).

Interprétation : Limiter la profondeur (`max_depth=3`) agit comme un régularisateur. Laisser l'arbre croître indéfiniment réduit le biais sur les données d'entraînement au détriment d'une variance excessive sur le jeu de test.

```
from sklearn.tree import DecisionTreeRegressor
modele = DecisionTreeRegressor(max_depth=3, min_samples_leaf=8, random_state=GRAIN
print('MAE arbre :', round(mean_absolute_error(yr_test, modele.predict(Xr_test)),
```

Fermer

```
## 6. Forêt aléatoire — régression

**Variante C — question spécifique :** La

**À compléter :** Présentation (avec équat

> Modifiez au moins un hyperparamètre et j

### Réponses : 6. Forêt aléatoire — régres

**C1 — Présentation**
- **Catégorie :** Apprentissage supervisé
- **Entrées / Sorties :** Entrée $X \in \m
```

6. Forêt aléatoire — régression

Variante C — question spécifique : La forêt réduit-elle la variance par rapport à l'arbre ?

Réponses : 6. Forêt aléatoire — régression

C1 — Présentation

```
- **Hypothèse clé :** Agrégat d'arbres déc
- **Équation :** $\hat{y}_{RF}(x) = \frac{
  - $T_b(x)$ : prédiction du $b$-ième arbr
- **Paramètre appris / Hyperparamètre :**

**C2 — Schéma**
*(Génération de multiples échantillons boo

**C3 — Décider quand l'utiliser**
- **Contextes :** Actuariat (prédire le co
- **Structure :** Données tabulaires hétér
- **Concurrent :** Gradient Boosting ou SV
- **Métrique :** MAE.

**C4 — Test de sensibilité et interprétati
En analysant l'impact du nombre d'arbres (
- Pour **n_estimators = 10** : MAE Test =
- Pour **n_estimators = 60** : MAE Test =
- Pour **n_estimators = 150** : MAE Test =

*Interprétation :* Oui, la forêt aléatoire
```

- **Catégorie :** Apprentissage supervisé (Ensemble de régression par Bagging).
- **Entrées / Sorties :** Entrée $X \in \mathbb{R}^{n \times p}$; Sortie $y \in \mathbb{R}$ (moyenne des prédictions des arbres).
- **Hypothèse clé :** Agrégat d'arbres décorrélés (grâce au bootstrap et au sous-espace de variables) réduit considérablement la variance globale.
- **Équation :** $\hat{y}_{RF}(x) = \frac{1}{B} \sum_{b=1}^{B} T_b(x)$
  - $T_b(x)$: prédiction du b-ième arbre de décision entraîné sur un échantillon bootstrappé.
- **Paramètre appris / Hyperparamètre :** Ensemble des structures des arbres individuels / Nombre d'arbres `n_estimators`, profondeur `max_depth`.

C2 — Schéma *(Génération de multiples échantillons bootstrappés, entraînement d'arbres indépendants et agrégation finale par moyenne)*

C3 — Décider quand l'utiliser

- **Contextes :** Actuariat (prédire le coût des sinistres d'assurance en intégrant des profils complexes) ou Supply Chain (prévision de la demande produit sur de multiples entrepôts).
- **Structure :** Données tabulaires hétérogènes, pas besoin de mise à l'échelle des caractéristiques.
- **Concurrent :** Gradient Boosting ou SVR.
- **Métrique :** MAE.

C4 — Test de sensibilité et interprétation
En analysant l'impact du nombre d'arbres (`n_estimators`) :

- Pour **n_estimators = 10** : MAE Test = **44.67**
- Pour **n_estimators = 60** : MAE Test = **43.11**

- Pour **n_estimators = 150** : MAE Test = **42.91**

Interprétation : Oui, la forêt aléatoire réduit considérablement la variance. Alors qu'un arbre profond unique (`max_depth=10` ou `None`) avait une MAE de test désastreuse de **50.27**, l'agrégation de multiples arbres robustes ramène l'erreur de test à **42.91** (pour 150 arbres). Plus on ajoute d'arbres, plus la variance est lissée et l'erreur de généralisation diminue sans risque de surapprentissage.

```
from sklearn.ensemble import RandomForestRegressor
modele = RandomForestRegressor(n_estimators=60, max_depth=6, n_jobs=-1, random_sta
print('MAE forêt :', round(mean_absolute_error(yr_test, modele.predict(Xr_test)),
```

Fermer

```
## 7. Gradient Boosting — régression

**Variante C — question spécifique :** Que

**À compléter :** Présentation (avec équat

> Modifiez au moins un hyperparamètre et j

### Réponses : 7. Gradient Boosting — régr

**C1 — Présentation**
- **Catégorie :** Apprentissage supervisé
- **Entrées / Sorties :** Entrée $X \in \m
- **Hypothèse clé :** Correction successiv
- **Équation :** $F_m(x) = F_{m-1}(x) + \g
  - $F_m(x)$ : prédiction à l'étape $m$, $
- **Paramètre appris / Hyperparamètre :**

**C2 — Schéma**
*(Processus séquentiel : calcul des résidu

**C3 — Décider quand l'utiliser**
- **Contextes :** E-commerce (estimation d
- **Structure :** Variables de types mixte
- **Concurrent :** Random Forest ou LightG
- **Métrique :** MAE.

**C4 — Test de sensibilité et interprétati
En observant l'évolution selon les itérati
- **max_iter = 10** : MAE Entraînement = *
- **max_iter = 100** : MAE Entraînement =
- **max_iter = 300** : MAE Entraînement =
```

7. Gradient Boosting — régression

Variante C — question spécifique : Que change max_iter sur le coût et le risque de surajustement ?

Réponses : 7. Gradient Boosting — régression

C1 — Présentation

- **Catégorie :** Apprentissage supervisé (Ensemble de régression par Boosting séquentiel).
- **Entrées / Sorties :** Entrée $X \in \mathbb{R}^{n \times p}$; Sortie $y \in \mathbb{R}$.
- **Hypothèse clé :** Correction successive des résidus des modèles précédents par des arbres faibles ajustés sur le gradient du coût.
- **Équation :**
  $F_m(x) = F_{m-1}(x) + \gamma \cdot h_m(x)$
  - $F_m(x)$: prédiction à l'étape m, $h_m(x)$: arbre faible entraîné sur

```
*Interprétation :* `max_iter` contrôle la
```

les gradients négatifs, γ : taux d'apprentissage (shrinkage).

- **Paramètre appris / Hyperparamètre :** Structures et valeurs des feuilles des arbres successifs / Nombre de rounds de boosting `max_iter`.

C2 — Schéma *(Processus séquentiel : calcul des résidus à l'étape M, entraînement d'un arbre faible sur ces résidus, mise à jour des prédictions globales)*

C3 — Décider quand l'utiliser

- **Contextes :** E-commerce (estimation du panier moyen ou Customer Lifetime Value) ou Finance (modélisation de courbes de rendement de produits dérivés complexes).
- **Structure :** Variables de types mixtes, structures hautement non linéaires, tolérance au temps de calcul.
- **Concurrent :** Random Forest ou LightGBM.
- **Métrique :** MAE.

C4 — Test de sensibilité et interprétation
En observant l'évolution selon les itérations de boosting (`max_iter`) :

- **max_iter = 10** : MAE Entraînement = **46.24** | MAE Test = **48.19** (Sous-ajustement, modèle pas assez complexe).
- **max_iter = 100** : MAE Entraînement = **21.42** | MAE Test = **44.18** (Excellent équilibre biais-variance, meilleure MAE de test).
- **max_iter = 300** : MAE Entraînement = **9.00** | MAE Test = **46.75** (Surajustement prononcé : le coût d'entraînement s'effondre mais la MAE de test remonte).

Interprétation : `max_iter` contrôle la complexité globale du modèle. Un nombre trop élevé conduit au surapprentissage car

> le modèle finit par apprendre par cœur le bruit des données d'entraînement.

```
from sklearn.ensemble import HistGradientBoostingRegressor
modele = HistGradientBoostingRegressor(max_iter=100, max_leaf_nodes=9, random_stat
print('MAE boosting :', round(mean_absolute_error(yr_test, modele.predict(Xr_test)
```

8. Régression logistique

Variante C — question spécifique : Quel effet de C sur les probabilités et la frontière ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.linear_model import LogisticRegression
modele = make_pipeline(StandardScaler(), LogisticRegression(C=.5, max_iter=2000, r
print('F1 logistique :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

```
# Exécution de la régression logistique (Question 8)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
for c_val in [0.01, 0.5, 10.0]:
    modele_lr = make_pipeline(StandardScaler(), LogisticRegression(C=c_val, max_it
    f1_test = f1_score(yc_test, modele_lr.predict(Xc_test))
    print(f"Régression Logistique C = {c_val:<5} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de KNN (Question 9)
from sklearn.neighbors import KNeighborsClassifier
for k_val in [3, 7, 15]:
    modele_knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=
    f1_test = f1_score(yc_test, modele_knn.predict(Xc_test))
    print(f"KNN k = {k_val:<2} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de Naive Bayes (Question 10)
from sklearn.naive_bayes import GaussianNB
for smooth in [1e-9, 1e-5, 1e-1]:
    modele_nb = GaussianNB(var_smoothing=smooth).fit(Xc_ent, yc_ent)
    f1_test = f1_score(yc_test, modele_nb.predict(Xc_test))
    print(f"GaussianNB smoothing = {smooth:<5} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de l'arbre de décision (Question 11)
from sklearn.tree import DecisionTreeClassifier
modele_tree = DecisionTreeClassifier(max_depth=3, min_samples_leaf=6, random_state
premiere_carac = jeu_classification.feature_names[modele_tree.tree_.feature[0]]
print(f"La caractéristique à la racine de l'arbre est : {premiere_carac}")
for prof in [2, 3, 10]:
    m_tree = DecisionTreeClassifier(max_depth=prof, min_samples_leaf=6, random_sta
    print(f"Arbre max_depth = {prof} | F1 Test = {f1_score(yc_test, m_tree.predict
```

```
# Exécution de la forêt aléatoire de classification (Question 12)
from sklearn.ensemble import RandomForestClassifier
for n_est in [10, 70, 200]:
    modele_rf = RandomForestClassifier(n_estimators=n_est, max_features='log2', ra
    f1_test = f1_score(yc_test, modele_rf.predict(Xc_test))
    print(f"Forêt n_estimators = {n_est:<3} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de SVM (Question 13)
from sklearn.svm import SVC
for noyau in ['linear', 'rbf']:
    modele_svm = make_pipeline(StandardScaler(), SVC(kernel=noyau, C=0.2, random_s
    f1_test = f1_score(yc_test, modele_svm.predict(Xc_test))
    print(f"SVM noyau = {noyau:<7} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de HistGradientBoosting pour classification (Question 14)
from sklearn.ensemble import HistGradientBoostingClassifier
import time
for max_it in [10, 65, 200]:
    debut = time.time()
    modele_gb = HistGradientBoostingClassifier(max_iter=max_it, max_leaf_nodes=8,
    duree = time.time() - debut
    f1_test = f1_score(yc_test, modele_gb.predict(Xc_test))
    print(f"GB max_iter = {max_it:<3} | Temps = {duree:.4f}s | F1 Test = {f1_test:
```

```
# Exécution d'AdaBoost (Question 15)
from sklearn.ensemble import AdaBoostClassifier
for lr in [0.05, 0.6, 2.0]:
    modele_ada = AdaBoostClassifier(n_estimators=50, learning_rate=lr, random_stat
    f1_test = f1_score(yc_test, modele_ada.predict(Xc_test))
    print(f"AdaBoost learning_rate = {lr:<4} | F1 Test = {f1_test:.3f}")
```

Gemini

```
# Exécution de XGBoost (Question 16)
from xgboost import XGBClassifier
for prof in [1, 2, 6]:
    modele_xgb = XGBClassifier(n_estimators=65, max_depth=prof, learning_rate=0.08
    f1_test = f1_score(yc_test, modele_xgb.predict(Xc_test))
    print(f"XGBoost max_depth = {prof} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de la régression logistique (Question 8)
from sklearn.linear_model import LogisticRegression
for c_val in [0.01, 0.5, 10.0]:
    modele_lr = make_pipeline(StandardScaler(), LogisticRegression(C=c_val, max_it
    f1_test = f1_score(yc_test, modele_lr.predict(Xc_test))
    print(f"Régression Logistique C = {c_val:<5} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de KNN (Question 9)
from sklearn.neighbors import KNeighborsClassifier
for k_val in [3, 7, 15]:
    modele_knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=
    f1_test = f1_score(yc_test, modele_knn.predict(Xc_test))
    print(f"KNN k = {k_val:<2} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de Naive Bayes (Question 10)
from sklearn.naive_bayes import GaussianNB
for smooth in [1e-9, 1e-5, 1e-1]:
    modele_nb = GaussianNB(var_smoothing=smooth).fit(Xc_ent, yc_ent)
    f1_test = f1_score(yc_test, modele_nb.predict(Xc_test))
    print(f"GaussianNB smoothing = {smooth:<5} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de l'arbre de décision (Question 11)
from sklearn.tree import DecisionTreeClassifier
modele_tree = DecisionTreeClassifier(max_depth=3, min_samples_leaf=6, random_state
# Afficher la caractéristique la plus discriminante (à la racine)
premiere_carac = jeu_classification.feature_names[modele_tree.tree_.feature[0]]
print(f"La caractéristique à la racine de l'arbre est : {premiere_carac}")
for prof in [2, 3, 10]:
    m_tree = DecisionTreeClassifier(max_depth=prof, min_samples_leaf=6, random_sta
    print(f"Arbre max_depth = {prof} | F1 Test = {f1_score(yc_test, m_tree.predict
```

```
# Exécution de la forêt aléatoire de classification (Question 12)
from sklearn.ensemble import RandomForestClassifier
for n_est in [10, 70, 200]:
    modele_rf = RandomForestClassifier(n_estimators=n_est, max_features='log2', ra
    f1_test = f1_score(yc_test, modele_rf.predict(Xc_test))
    print(f"Forêt n_estimators = {n_est:<3} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de SVM (Question 13)
from sklearn.svm import SVC
for noyau in ['linear', 'rbf']:
    modele_svm = make_pipeline(StandardScaler(), SVC(kernel=noyau, C=0.2, random_s
    f1_test = f1_score(yc_test, modele_svm.predict(Xc_test))
    print(f"SVM noyau = {noyau:<7} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de HistGradientBoosting pour classification (Question 14)
from sklearn.ensemble import HistGradientBoostingClassifier
import time
for max_it in [10, 65, 200]:
    debut = time.time()
    modele_gb = HistGradientBoostingClassifier(max_iter=max_it, max_leaf_nodes=8,
    duree = time.time() - debut
    f1_test = f1_score(yc_test, modele_gb.predict(Xc_test))
    print(f"GB max_iter = {max_it:<3} | Temps = {duree:.4f}s | F1 Test = {f1_test:
```

```
# Exécution d'AdaBoost (Question 15)
from sklearn.ensemble import AdaBoostClassifier
for lr in [0.05, 0.6, 2.0]:
    modele_ada = AdaBoostClassifier(n_estimators=50, learning_rate=lr, random_stat
    f1_test = f1_score(yc_test, modele_ada.predict(Xc_test))
    print(f"AdaBoost learning_rate = {lr:<4} | F1 Test = {f1_test:.3f}")
```

```
# Exécution de XGBoost (Question 16)
from xgboost import XGBClassifier
for prof in [1, 2, 6]:
    modele_xgb = XGBClassifier(n_estimators=65, max_depth=prof, learning_rate=0.08
    f1_test = f1_score(yc_test, modele_xgb.predict(Xc_test))
    print(f"XGBoost max_depth = {prof} | F1 Test = {f1_test:.3f}")
```

9. KNN

Variante C — question spécifique : Comparez k = 3, 7 et 15 ; quelle métrique varie ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.neighbors import KNeighborsClassifier
modele = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7, weigh
print('F1 KNN :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

10. Naive Bayes

Variante C — question spécifique : Dans quel cas l'hypothèse d'indépendance devient-elle critique ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.naive_bayes import GaussianNB
modele = GaussianNB(var_smoothing=1e-8).fit(Xc_ent, yc_ent)
print('F1 Naive Bayes :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

11. Arbre de décision

Variante C — question spécifique : Quelle variable guide la première séparation de l'arbre ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.tree import DecisionTreeClassifier
modele = DecisionTreeClassifier(max_depth=3, min_samples_leaf=6, random_state=GRAI
print('F1 arbre :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

12. Forêt aléatoire — classification

Variante C — question spécifique : Quel effet du nombre d'arbres sur la stabilité ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.ensemble import RandomForestClassifier
modele = RandomForestClassifier(n_estimators=70, max_features='log2', random_state=0
print('F1 forêt :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

13. SVM

Variante C — question spécifique : Comparez noyau linéaire et RBF, et justifiez le choix.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.svm import SVC
modele = make_pipeline(StandardScaler(), SVC(kernel='linear', C=.2, random_state=GRA
print('F1 SVM linéaire :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

14. Gradient Boosting — classification

Variante C — question spécifique : Quel effet du nombre d'itérations sur le temps et F1 ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.ensemble import HistGradientBoostingClassifier
modele = HistGradientBoostingClassifier(max_iter=65, max_leaf_nodes=8, random_stat
print('F1 boosting :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

15. AdaBoost

Variante C — question spécifique : Que change learning_rate quand n_estimators reste fixe ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.ensemble import AdaBoostClassifier
modele = AdaBoostClassifier(n_estimators=50, learning_rate=.6, random_state=GRAINE
print('F1 AdaBoost :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

16. XGBoost

Variante C — question spécifique : Que change max_depth sur la capacité du modèle ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from xgboost import XGBClassifier
modele = XGBClassifier(n_estimators=65, max_depth=2, learning_rate=.08, random_sta
print('F1 XGBoost :', round(f1_score(yc_test, modele.predict(Xc_test)), 3))
```

17. K-means

Variante C — question spécifique : Évaluez k = 2, 3, 4 avec silhouette et inertie.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.cluster import KMeans
modele = KMeans(n_clusters=3, n_init=15, random_state=GRAINE).fit(X_clu)
print('Inertie K-means :', round(modele.inertia_, 2))
print('Silhouette :', round(silhouette_score(X_clu, modele.labels_), 3))
```

18. Classification hiérarchique

Variante C — question spécifique : Comparez complete et ward sur les mêmes données.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.cluster import AgglomerativeClustering
etiquettes = AgglomerativeClustering(n_clusters=3, linkage='complete').fit_predict
print('Silhouette CHA (complete) :', round(silhouette_score(X_clu, etiquettes), 3)
```

19. DBSCAN

Variante C — question spécifique : Expliquez pourquoi eps modifie le nombre de points bruit.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.cluster import DBSCAN
etiquettes = DBSCAN(eps=2.5, min_samples=4).fit_predict(X_clu)
print('Groupes détectés hors bruit :', len(set(etiquettes) - {-1}))
print('Points considérés comme bruit :', np.sum(etiquettes == -1))
```

20. Mélanges gaussiens

Variante C — question spécifique : Comparez covariance_type = full et diag.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.mixture import GaussianMixture
modele = GaussianMixture(n_components=3, covariance_type='diag', random_state=GRAI
etiquettes = modele.predict(X_clu)
print('Poids du mélange :', np.round(modele.weights_, 3))
print('Silhouette :', round(silhouette_score(X_clu, etiquettes), 3) if len(set(eti
```

21. ACP

Variante C — question spécifique : Combien de composantes faut-il pour 80 % de variance ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.decomposition import PCA
modele = PCA(n_components=3, random_state=GRAINE).fit(X_clu)
projection = modele.transform(X_clu)
print('Variance expliquée par composante :', np.round(modele.explained_variance_ra
plt.scatter(projection[:, 0], projection[:, 1], s=18)
plt.title('Wine — projection ACP sans étiquettes'); plt.show()
```

22. t-SNE

Variante C — question spécifique : La structure change-t-elle quand perplexity passe de 12 à 30 ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.manifold import TSNE
projection = TSNE(n_components=2, perplexity=12, learning_rate='auto', init='pca',
plt.scatter(projection[:, 0], projection[:, 1], s=18)
plt.title('Wine — t-SNE (perplexity=12)'); plt.show()
```

23. UMAP

Variante C — question spécifique : Que provoque n_neighbors = 8 puis 30 ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
import umap
projection = umap.UMAP(n_components=2, n_neighbors=8, min_dist=.25, random_state=G
plt.scatter(projection[:, 0], projection[:, 1], s=18)
plt.title('Wine — UMAP (n_neighbors=8)'); plt.show()
```

24. Isolation Forest

Variante C — question spécifique : Comparez contamination = 0.03 et 0.10.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
from sklearn.ensemble import IsolationForest
modele = IsolationForest(contamination=.07, n_estimators=120, random_state=GRAINE)
etiquettes = modele.predict(X_clu)
print('Observations atypiques :', int((etiquettes == -1).sum()))
print('Attention : atypique ne signifie pas erroné ou frauduleux.')
```

25. Bandits manchots

Variante C — question spécifique : Quel bras obtient la meilleure moyenne après 800 tirages ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
# Bandit pédagogique simulé : estimation des probabilités de gain
rng = np.random.default_rng(GRAINE)
probabilites_simulees = np.array([.35, .55, .65, .45])
recompenses = np.zeros(4); nombre_tirages = np.zeros(4)
for etape in range(800):
    bras = etape % 4
```

```
    gain = float(rng.random() < probabilites_simulees[bras])
    recompenses[bras] += gain; nombre_tirages[bras] += 1
print('Récompenses moyennes estimées :', np.round(recompenses / nombre_tirages, 3)
```

26. Epsilon-greedy

Variante C — question spécifique : Quel effet de epsilon sur la concentration des choix ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
# Exploration epsilon-greedy sur le même bandit simulé
rng = np.random.default_rng(GRAINE)
probabilites_simulees = np.array([.35, .55, .65, .45])
recompenses = np.zeros(4); nombre_tirages = np.zeros(4)
epsilon = .2
for etape in range(1500):
    bras = int(rng.integers(4)) if rng.random() < epsilon or not np.all(nombre_tir
    gain = float(rng.random() < probabilites_simulees[bras])
    recompenses[bras] += gain; nombre_tirages[bras] += 1
print('Fréquences de sélection :', nombre_tirages.astype(int))
print('Valeurs estimées :', np.round(recompenses / np.maximum(nombre_tirages, 1),
```

27. Q-learning

Variante C — question spécifique : Étudiez les récompenses des épisodes, pas seulement Q moyen.

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
# Q-learning manuel — environnement CliffWalking (Gymnasium)
import gymnasium as gym
env = gym.make('CliffWalking-v0')
q_table = np.zeros((env.observation_space.n, env.action_space.n))
rng = np.random.default_rng(GRAINE)
for episode in range(1100):
    etat, _ = env.reset(seed=GRAINE + episode)
    epsilon = max(.08, .8 * (1 - episode / 900))
    for pas in range(120):
        action = int(rng.integers(env.action_space.n)) if rng.random() < epsilon e
        suivant, recompense, termine, tronque, _ = env.step(action)
        cible = recompense + (0 if termine else .95 * np.max(q_table[suivant]))
        q_table[etat, action] += .15 * (cible - q_table[etat, action])
        etat = suivant
```

```
            if termine or tronque: break
    print('Table Q :', q_table.shape, '| valeur moyenne :', round(q_table.mean(), 2))
    env.close()
```

28. SARSA

Variante C — question spécifique : SARSA privilégie-t-il une trajectoire plus sûre sous exploration ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
    # SARSA manuel — même environnement, politique epsilon-greedy suivie
    import gymnasium as gym
    env = gym.make('CliffWalking-v0')
    q_table = np.zeros((env.observation_space.n, env.action_space.n))
    rng = np.random.default_rng(GRAINE)
    def choisir_action(etat, epsilon):
        return int(rng.integers(env.action_space.n)) if rng.random() < epsilon else in
    for episode in range(1100):
        etat, _ = env.reset(seed=GRAINE + episode)
        epsilon = max(.08, .8 * (1 - episode / 900))
        action = choisir_action(etat, epsilon)
        for pas in range(120):
            suivant, recompense, termine, tronque, _ = env.step(action)
            action_suivante = choisir_action(suivant, epsilon)
            cible = recompense + (0 if termine else .95 * q_table[suivant, action_suiv
            q_table[etat, action] += .15 * (cible - q_table[etat, action])
            etat, action = suivant, action_suivante
            if termine or tronque: break
    print('Table Q SARSA :', q_table.shape, '| valeur moyenne :', round(q_table.mean()
    env.close()
```

29. DQN — cible conceptuelle

Variante C — question spécifique : Pourquoi la cible est-elle égale à la récompense en état terminal ?

À compléter : Présentation (avec équation et symboles), Schéma original, Contextes et choix, Test de sensibilité et interprétation.

> Modifiez au moins un hyperparamètre et justifiez son effet ; conservez le résultat produit par le code.

```
    # Cibles DQN : démonstration du calcul de Bellman, SANS réseau neuronal
    recompenses = np.array([-1., -100., 0.])
    termine = np.array([False, True, False])
    meilleure_valeur_suivante = np.array([-3., -2., 5.])
```

```
gamma = .9
cibles = recompenses + gamma * (~termine) * meilleure_valeur_suivante
print('Cibles de Bellman calculées :', np.round(cibles, 2))
print('Ce calcul ne constitue pas un entraînement DQN.')
```

Annexes — implémentations manuelles, variante C

En complément des modèles intégrés : (1) descente de gradient univariée sur une variable réelle de Diabetes, (2) K-means manuel sur la projection ACP de Wine, (3) Q-learning tabulaire manuel dans l'exemple 27. Les modèles manuels sont à comparer aux références scikit-learn **sur les mêmes données**, si une référence existe.

```
# Descente de gradient manuelle sur Diabetes (une seule caractéristique)
from sklearn.linear_model import LinearRegression
x = Xr_ent[:, 2].copy()  # Indice de masse corporelle, caractéristique déjà centré
x = (x - x.mean()) / x.std()
y = yr_ent.copy()
poids, biais, taux = 0., 0., .05
for iteration in range(1800):
    erreur = (poids * x + biais) - y
    poids -= taux * 2 * np.mean(erreur * x)
    biais -= taux * 2 * np.mean(erreur)
reference = LinearRegression().fit(x.reshape(-1, 1), y)
print('Manuel :', round(poids, 3), round(biais, 3))
print('scikit-learn :', round(reference.coef_[0], 3), round(reference.intercept_,
```